



DSA

Module 3

Bit Manipulation

Author
Srinivas Dande



My **JAVA
LEARNING
CENTER**



3. Bitwise Operators

- ◆ These Operators will be used to perform the operation on individual bits of the value.

Bitwise Operators	Description
~	Bitwise NOT
&	Bitwise AND
	Bitwise OR
^	Exclusive OR (XOR)
<<	Left Shift
>>	Right Shift
>>>	Unsigned Right Shift

- ◆ Operands for the following Bitwise Operators can be of boolean type or Integer type:
 - Bitwise AND
 - Bitwise OR
 - Exclusive OR (XOR)
- ◆ Operands for the following Bitwise Operators will be of Integer type only:
 - Bitwise NOT
 - Left Shift
 - Right Shift
 - Unsigned Right Shift

1's Complement

- ◆ 1's complement of a binary number is formed by changing 1's to 0's and 0's to 1's

0	0	1	0	1	1	0	1	Actual Bits
1	1	0	1	0	0	1	0	1's Complement



2's Complement

- ♦ 2's complement of a binary number is formed by adding 1 to the least significant bit of 1's complement.

0	0	1	0	1	1	0	1	Actual Bits
1	1	0	1	0	0	1	0	1's Complement
							1	Adding 1 with value
1	1	0	1	0	0	1	1	2's Complement

3.1. Exploring Bitwise AND

- ♦ Bitwise AND returns 1 when both the bits are 1
- ♦ Bitwise AND returns 0 when any one bit is 0.
- ♦ **Truth Table of Bitwise AND**

A	B	A & B
1	1	1
1	0	0
0	1	0
0	0	0

Lab1.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab1 {
    public static void main(String[] args) {
        int a= 5;
        int b= 7;
        int result= a & b;
        System.out.println(result);
    }
}
```



3.2. Exploring Bitwise OR

- ◆ Bitwise OR returns 0 when both the bits are 0
- ◆ Bitwise OR returns 1 when any one bit is 1.
- ◆ **Truth Table of Bitwise OR**

A	B	A B
1	1	1
1	0	1
0	1	1
0	0	0

Lab2.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {
    public static void main(String[] args) {
        int a= 5;
        int b= 7;

        int result= a | b;

        System.out.println(result);
    }
}
```



3.3. Exploring Bitwise XOR

- ◆ Bitwise XOR returns 0 when both the bits are same
- ◆ Bitwise XOR returns 1 when both the bits are different.
- ◆ **Truth Table of Bitwise XOR**

A	B	A ^ B
1	1	0
1	0	1
0	1	1
0	0	0

- ◆ **Imp Points to Remember with Bitwise XOR**

1. $(a \wedge b) \wedge c = a \wedge (b \wedge c)$
2. $a \wedge 0 = a$
3. $a \wedge a = 0$
4. $a \wedge b = b \wedge a$

Lab3.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab3 {
    public static void main(String[] args) {
        int a= 5;
        int b= 7;

        int result= a ^ b;

        System.out.println(result);
    }
}
```



Lab4.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab4 {
    public static void main(String[] args) {

        int p= 5;

        System.out.println(p ^ p); //0
        System.out.println(p ^ 0); //p
        System.out.println(0 ^ p); //p

        int a=9;
        int b= 13;
        int c= 25;

        int x= a^b;
        int y= b^a;

        System.out.println(x);
        System.out.println(y);

        x= (a^b)^c;
        y= a^(b^c);

        System.out.println(x);
        System.out.println(y);

    }
}
```



3.4. Exploring Bitwise NOT

- ◆ Bitwise NOT returns 0 when the Bit is 1
- ◆ Bitwise NOT returns 1 when the Bit is 0.
- ◆ **Truth Table of Bitwise NOT**

A	~A
1	0
0	1

Lab5.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab5 {

    public static void main(String[] args) {

        int a=8;
        int b= -7;

        int result1= ~a;
        int result2= ~b;

        System.out.println(result1);
        System.out.println(result2);

    }
}
```



3.5. Exploring Bitwise LeftShift

- ◆ Bitwise LeftShift Operator shift the Bits towards leftside for Specified number of Times
- ◆ When Bits are Shifted to Left Once then It is Equals to multiplying the number by 2

Lab6.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab6 {

    public static void main(String[] args) {

        int a=5;

        System.out.println(a<<1);
        System.out.println(a<<2);
        System.out.println(a<<3);
        System.out.println(a<<4);

    }

}
```



3.6. Exploring Bitwise Rightshift

- ◆ Bitwise RightShift Operator shift the Bits towards rightside for Specified number of Times
- ◆ When Bits are Shifted to Right Once then It is Equals to dividing the number by 2
- ◆ RightShift also called as Signed RightShift
- ◆ Signed RightShift holds the Sign i.e
 - If the Number is Positive then fills with **Zero**
 - If the Number is Negative then fills with **One**

Lab7.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab7 {

    public static void main(String[] args) {

        int a=25;

        System.out.println(a>>1);
        System.out.println(a>>2);

        int b=-25;

        System.out.println(b>>1);
        System.out.println(b>>2);

    }

}
```



3.7. Exploring Bitwise Unsigned Rightshift

- ◆ Bitwise Unsigned RightShift Operator shift the Bits towards rightside for Specified number of Times
- ◆ When Bits are Shifted to Right Once then It is Equals to dividing the number by 2
- ◆ Unsigned Signed RightShift Ignores the Sign i.e
 - If the Number is Positive then fills with **Zero**
 - If the Number is Negative then also fills with **Zero**

Lab8.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab8 {

    public static void main(String[] args) {

        int a=5;

        System.out.println(a>>>1);
        System.out.println(a>>>2);

        int b=-5;

        System.out.println(b>>>1);
        System.out.println(b>>>2);

    }

}
```



3.8. Problems on Bitwise

Problem1.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Check whether Last bit of given number is Set or Not
public class Problem1 {
    public static void main(String[] args) {
        int n=4;
        if((n & 1) != 0)
            System.out.println("Last Bit is Set");
        else
            System.out.println("Last Bit is Not Set");
    }
}
```

Probme2.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Check whether given number is Odd or Even
//If Last Bit is Set then It is Odd
//If Last Bit is Un-Set then It is Even
public class Problem2{
    public static void main(String[] args) {
        int n=6;
        if((n&1) !=0)
            System.out.println("Odd");
        else
            System.out.println("Even");
    }
}
```



Problem3A.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Check whether Kth bit of given number is Set or Not

public class Problem3A {

    public static void main(String[] args) {

        int n=15;
        int k=5;

        for(int i=1;i<=k-1;i++)
            n= n >> 1; //Shifting One time

        if((n & 1) != 0)
            System.out.println("Kth Bit is Set");
        else
            System.out.println("Kth Bit is Not Set");

    }

}

// Time Complexity => O(n)
```



Problem3B.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Check whether Kth bit of given number is Set or Not

public class Problem3B{

    public static void main(String[] args) {

        int n=15;
        int k=5;

        n = n >> k-1; //Shifting k-1 times

        if((n & 1) != 0)
            System.out.println("Kth Bit is Set");
        else
            System.out.println("Kth Bit is Not Set");

    }

}

// Time Complexity=> O(1)
```



Problem4A.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Count the Number of Set Bits in given number
// Visiting All the Bits and Counting only Set Bits

public class Problem4A {
    public static void main(String[] args) {

        int n = 1024;
        int count = 0;
        int loopCount = 0;

        while (n > 0) {
            loopCount++;
            if ((n & 1) != 0)
                count++;

            n = n >> 1; // n = n/2;
        }

        System.out.println("Loop Count : " + loopCount);
        System.out.println("Set Bit Count : " + count);
    }
}

// Time Complexity => O(100) => O(1)
```



Problem4B.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Count the Number of Set Bits in given number
//Brain Kerningam's Algorithm
//Visit and Count only Set Bits

public class Problem4B{
    public static void main(String[] args) {

        int n = 40;
        int count = 0;

        while (n > 0) {

            n = n & (n-1);
            count++;

        }

        System.out.println("Set Bit Count : " + count);
    }
}

// Time Complexity => O(1)
```



Problem5A.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Find whether Given Number is Power of 2 or Not
// Number of Set Bits are 1

public class Problem5A {
    public static void main(String[] args) {

        int n = 63;
        int count = 0;

        while (n > 0) {
            n = n & (n-1);
            count++;
        }

        System.out.println("Set Bit Count : " + count);

        if(count==1)
            System.out.println("2 Power ");
        else
            System.out.println("Not 2 Power ");

    }
}

// Time Complexity => O(n) / O(1)
```



Problem5B.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Find whether Given Number is Power of 2 or Not
// Number of Set Bits are 1

public class Problem5B{
    public static void main(String[] args) {

        int n = 64;

        if( ( n & (n-1) ) == 0)
            System.out.println("2 Power ");
        else
            System.out.println("Not 2 Power ");

    }
}

// Time Complexity => O(1)
```



LeetCode Bitwise Problems

1	LC-191 – Count 1 bits
2	LC-231 – Power of 2
3	LC-190 – Reverse Bits
4	LC-136 – Single Number
5	LC-260 – Single Number -III
6	LC-201 – Bitwise AND of number Range
7	LC-693 – Binary Number with Alternating Bits
8	LC-476 – Number Complement



LeetCode191.java

```
package com.myjlc.bitwise.problems;
```

```
// Count 1 Bits
```

```
//Brain Kerningam's Algorithm
```

```
public class LeetCode191 {  
  
    public static int hammingWeight(int n) {  
        int count = 0;  
        while (n!=0) {  
            n = n & (n - 1);  
            count++;  
        }  
        return count;  
    }  
    public static void main(String[] args) {  
        int count = hammingWeight(15);  
        System.out.println(count);  
    }  
}
```



LeetCode231.java

```
package com.myjlc.bitwise.problems;
```

```
//231. Power of Two
```

```
//Brain Kerningam's Algorithm
```

```
class LeetCode231 {  
  
    public static boolean isPowerOfTwo(int n) {  
        return n>0 && (n & (n - 1)) == 0;  
    }  
  
    public static void main(String[] args) {  
        boolean flag = isPowerOfTwo(-2147483648);  
        System.out.println(flag);  
    }  
}
```



LeetCode190.java

```
package com.myjlc.bitwise.problems;
```

```
//Reverse the bits of Given Number
```

```
public class LeetCode190 {

    public static int reverseBits(int num) {
        int result = 0;
        for (int i = 1; i <= 32; i++) {
            result = result << 1;

            if ((num & 1) > 0)
                result = result + 1;
            num = num >> 1;
        }
        return result;
    }

    public static void main(String[] args) {

        int x = reverseBits(13);
        System.out.println(x);
    }
}
```



LeetCode136.java

```
package com.myjlc.bitwise.problems;
```

```
//136. Single Number
```

```
//Find One Odd Occurring Number in the Array
```

```
public class LeetCode136 {  
    public static int singleNumber(int[] arr) {  
  
        int result = 0;  
  
        for (int i = 0; i < arr.length; i++) {  
            result = result ^ arr[i];  
        }  
        return result;  
    }  
  
    public static void main(String[] args) {  
        int arr[] = { 3, 5, 5, 6, 6, 7, 8, 8, 3 };  
        int result = singleNumber(arr);  
        System.out.println(result);  
    }  
}
```



LeetCode260.java

```
package com.myjlc.bitwise.problems;
```

```
//260. Single Number III
```

```
//Find Two Odd Occurring Numbers in the Array
```

```
public class LeetCode260 {  
    public static int[] singleNumber(int[] arr) {  
        int x = 0;  
        for (int i = 0; i < arr.length; i++) {  
            x = x ^ arr[i];  
        }  
  
        int k = x & ~(x - 1);  
  
        int result[] = new int[2];  
        for (int i = 0; i < arr.length; i++) {  
            if ((arr[i] & k) != 0)  
                result[0] = result[0] ^ arr[i];  
            else  
                result[1] = result[1] ^ arr[i];  
        }  
        return result;  
    }  
    public static void main(String[] args) {  
        int arr[] = { 3, 5, 9, 6, 6, 7, 2, 7, 2, 3 };  
        int result[] = singleNumber(arr);  
        for (int x : result) {  
            System.out.println(x);  
        }  
    }  
}
```



LeetCode201.java

```
package com.myjlc.bitwise.problems;
```

// 201. Bitwise AND of Numbers Range

```
public class LeetCode201 {  
    public static int rangeBitwiseAnd(int m, int n) {  
        int count = 0;  
        while(m!=n){  
            m >>= 1;  
            n >>= 1;  
            count ++;  
        }  
        return m << count;  
    }  
    public static void main(String[] args) {  
  
        int result = rangeBitwiseAnd(5,7);  
        System.out.println(result);  
  
    }  
}
```



LeetCode693.java

```
package com.myjlc.bitwise.problems;
```

//693. Binary Number with Alternating Bits

```
public class LeetCode693 {  
    public static boolean hasAlternatingBits(int num) {  
        int prevBit = num&1;  
        num = num>>1;  
  
        while(num>0) {  
            int currBit = num&1;  
            if(currBit==prevBit){ return false; }  
  
            prevBit = currBit;  
            num = num>>1;  
        }  
        return true;  
    }  
  
    public static void main(String[] args) {  
  
        boolean result = hasAlternatingBits(4);  
        System.out.println(result);  
    }  
}
```



LeetCode476.java

```
package com.myjlc.bitwise.problems;
```

//476. Number Complement

```
public class LeetCode476 {  
  
    public static int findComplement(int num) {  
        int result=0;  
        int i=0;  
        while(num!=0) {  
            if((num & 1) ==0) {  
                result = result + (1<<i);  
            }  
            i++;  
            num =num >>1;  
        }  
        return result;  
    }  
  
    public static void main(String[] args) {  
  
        int result = findComplement(5);  
        System.out.println(result);  
    }  
}
```